



Chapter 4

Object State

Concepts and Principles: Object equality, object identity, object life-cycle, object state (abstract vs. concrete), object uniqueness, optional types, state diagram, state space;

Patterns and Antipatterns: SPECULATIVE GENERALITY[†], TEMPORARY FIELD[†], LONG METHOD[†], NULL OBJECT, FLYWEIGHT, SINGLETON.

One of the most difficult things to reason about when looking at a program is state changes. Which operations can have a side-effect? On which path can data flow? What impacts what? This chapter clarifies what object state is and how we can manage to keep control over its state in a principled way.

Design Context

In this chapter, I continue the discussion of how to design abstractions to effectively represent a deck of cards in code (see the Design Context section of Chapter 3 for details).

4.1 The Static and Dynamic Perspectives of a Software System

There are different ways we can look at a software system. One way is in terms of the software elements declared in the source code and the relations between them. For example, a `Deck` class declares a field `aCards` that is a list of `Card` instances. This is a *static* (or compile-time) perspective of the system. The static perspective is best represented by the source code or a class diagram. A different, but complementary, way to look at a system, is in terms of objects in a running software system. For

example, at one point a `Deck` instance contains three cards, then one card is drawn, which leads to the instance of `Deck` containing two cards, etc. This is the *dynamic* (or *run-time*) perspective on the software. The dynamic perspective corresponds to the set of all values and references held by the variables in a program at different points in time. It is what we see in a debugger while stepping through the execution of the code. The dynamic perspective cannot easily be represented by any one diagram. Instead, we rely on object diagrams, state diagrams (introduced in this chapter), and sequence diagrams (introduced in Chapter 6). The static and dynamic perspectives are complementary in software design. Sometimes it is best to think of a problem and solution in static terms, sometimes in dynamic terms, and sometimes we really need both. This duality between the static and dynamic perspectives on a software system is akin to the wave-particle duality for representing the phenomenon of light in physics:

It seems as though we must use sometimes the one theory and sometimes the other, while at times we may use either. [...] We have two contradictory pictures of reality; separately neither of them fully explains the phenomena of light, but together they do.

—Albert Einstein and Leopold Infeld, *The Evolution of Physics*

To paraphrase for software design: It seems as though we must use sometimes the one perspective and sometimes the other, while at times we may use either. We have two *complementary* pictures of a software system; separately neither of them fully explains the phenomena of software, but together they do. This chapter focuses on understanding important *dynamic* properties of software.

4.2 Defining Object State

An important concept when thinking of a design in terms of run-time objects is that of *object state*. Informally, the state of an object refers to the particular pieces of information the object represents at a given moment. It is generally useful to distinguish between *concrete state* and *abstract state*. The concrete state of an object is the collection of values stored in the object's fields. For example, we can consider a `Player` object which, for now, only holds a score for the player in a scored game of Solitaire:

```
public class Player {
    private int aScore = 0;
}
```

The cardinality of the set of possible concrete states for `Player` is the number of different values that its single field, `aScore`, can take. Because, in Java, a variable of type `int` is allocated 32 bits of storage, the number of different concrete states that an object of type `Player` can be in is 2^{32} , or about 4 billion states. We usually refer to the set of possible states for a variable or object as its *state space*. As soon as objects have more complex types, the size of the state space explodes dramatically. For example, the state space of a `Deck` instance includes all possible permutations

of any number of cards in the deck, a number in the range of 2.2×10^{68} . This is an enormous number.¹ With class `Player`, adding an `aName` field of type `String` blows up the size of the state space to something that is only limited by the computing environment. For this reason, when designing software, it is more practical to think in terms of *abstract states*.

An abstract state is an arbitrarily-defined subset of the concrete state space. For example, considering the simple version of `Player` without the `aName` field, *Even Score* could be an abstract state for a `Player` instance that groups the roughly 2^{31} states that represent a score that is an even number. Likewise, for an instance of the `Deck` class, the abstract state *Three Kings* could represent any possible configuration of the deck where exactly three cards with rank `KING` are present. These two examples illustrate that, because abstract states are *arbitrary* partitions of the state space, they can really be defined as anything, no matter how whimsical. However, neither of these two example abstract states would be particularly useful to design a realistic software system. In practice, the software design task of state space partitioning is to define abstract states that correspond to characteristics that will help construct a clean solution. A more useful abstract state for `Player` would be *Non-zero Score*, and one for `Deck` would be *Empty* (no cards in the deck), which in the latter case happens to correspond to a single concrete state. When the distinction is necessary, I use the term *meaningful* abstract state to indicate abstract states that capture states that impact how an object would be used. For example, the abstract state *Empty* is meaningful because it is not possible to draw a card from an empty deck. In contrast, the abstract state *Three Kings* is not meaningful because, at least in a game of Solitaire, whether a deck contains three kings or not has no impact on the game play and is not related to any design or implementation decision. Unless otherwise noted, future references to the term *abstract state* assume that we are talking about meaningful abstract states.

A special case when thinking about object state is that some objects do not store any values. For example, function objects (see Section 3.4), often do not have any fields besides constants. In this case, we talk about *stateless* objects. When the contrast is important, we can refer to objects that have state as *stateful* objects. Another property of objects that is related to their state is mutability (see Section 2.6). This chapter is concerned with objects that are both mutable and stateful. In the case of immutable objects, the boundary between statefulness and statelessness becomes blurry, because in practice they only have a single state.

4.3 State Diagrams

As the result of method calls, a stateful and mutable object may *transition* between different abstract states. The sequence of abstract states that an object transitions through can be referred to as its *life cycle*, because it describes the “life” of an

¹ Computed as $\sum_{k=0}^{52} \frac{52!}{(52-k)!}$.

object, from its initialization to its abandonment and eventual destruction by the garbage collector. As a simple example, we can consider an instance of a class `Deck` that represents a deck of playing cards. Let us assume that instances of the class are initially empty and that a method `shuffle()` initializes the instance with 52 cards in random order. In this case, the method call will induce a transition between an abstract state *Empty* and an abstract state *Complete* that represents a full deck. Following an uninterrupted sequence of 52 calls to a method `draw()`, which removes a card from the deck, the instance will return to the *Empty* abstract state.

UML *state diagrams*² are useful to represent how objects can, during their lifetime, transition from one abstract state to another as a reaction to external events (typically, method calls). They represent a *dynamic* view of a software system. The annotated diagram in Figure 4.1 shows all the state diagram notation used in the book.

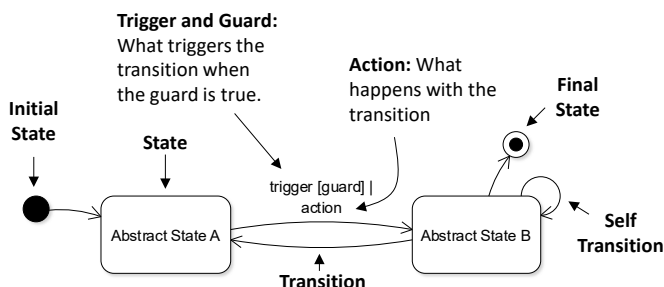


Fig. 4.1 Selected notation for state diagrams

The example in Figure 4.2 illustrates both the notation and purpose of UML state diagrams. It models some of the important abstract states of an instance of a class `Deck` as discussed above. Even this simple diagram captures key information about the design of the `Deck` class.

The object behavior modeled by the state diagram starts at the initial state. The initial state transitions into the *Empty* abstract state, which allows us to infer that the constructor returns a reference to a new `Deck` object with no cards in it. In UML, the initial and final states are special states that indicate the start and end of the state machine, respectively.³ Unlike regular states in the diagram, they do not represent a partition of the abstract state space. In our case, the transition out of the initial state corresponds to a call to the constructor of the class. The transitions are annotated with names that correspond to methods of the class `Deck`.

In a state diagram, *absence of a transition* usually means that the absent transition is not possible (i.e., invalid) for that state. In particular, a transition can only occur

² The official name of the diagram is *UML state machine diagram*. In this book I use the simpler form *state diagram*.

³ Technically the initial state is called a *pseudostate*, but this distinction is not important here.

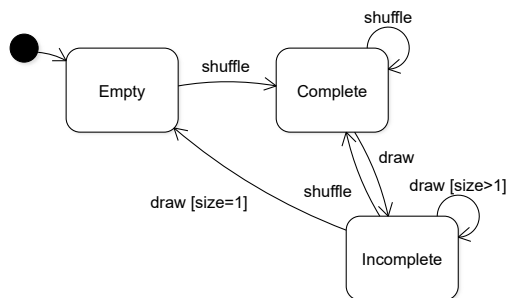


Fig. 4.2 State diagram for an instance of `Deck`

if all applicable preconditions are respected (see Section 2.9). Here we can see that we cannot `draw` cards from an empty deck, which happens to correspond to the existence of a precondition that the desk must not be empty for client code to call method `draw()`. Thus, the only legal transition out of the *Empty* state is *shuffle* which brings the object to the *Complete* state. From this it can be inferred that *Complete* is a shorthand for *Complete and shuffled* (in this particular design).

The *shuffle* transition out of the *Complete* state illustrates the idea of *self transitions*, namely, events that do not result in a change of abstract state. The only transition out of the *Complete* state is *draw*, which brings the deck object to an *Incomplete* state.

It is also possible to attach to a transition an *action* that describes what happens as the result of the transition. The action that corresponds to the *draw* event is *remove card from the deck*. The action information is optional and here I chose to leave it out of the diagram because it seemed redundant with the name of the event (considering that *to draw* is a synonym of *to remove* in the context of card games).

The two transitions out of the *Incomplete* state illustrate the importance of *guards*, because here without the concept of a guard we would not be able to model the distinction between a *draw* event that leads to the *Empty* state, and a *draw* event that keeps the object in the *Incomplete* state. The language I use for modeling guards does not follow a formal specification, but I nevertheless like to specify guards using pseudo-code that is very close to what could be reasonably tested on an instance of the object. Here the guard would assume the presence of a `size()` method in the `Deck` class.

Finally, the diagram in Figure 4.2 does not include any final state. While the meaning of the initial state is the context in which an object is initialized, the meaning of the final state is open to interpretation. One possible use of the final state is to indicate the state in which an object is expected to be at the end of its lifetime. In many designs, objects can end their life (stop being used) in any state. In this latter case, the final state model element can be omitted.

When getting started with modeling object states with state diagrams, one tendency is to use the state diagram notation to model a type of data flow information, where states represent processing, and arrows represent the flow of data between

processing stages. This is an incorrect use of the notation.⁴ A tip to avoid this pitfall is to think about the names of the states. If the names assigned to states include verbs or feel like they are describing actions (for example, *draw card*), it can be a sign that the diagram does not represent a good model of the state space.

State diagrams help us think about the state space for objects of a given class systematically. When modeling the state of an object, a good practice is to visit each state and consider each possible type of transition. This procedure helps avoid overlooking certain paths through the code (e.g., shuffling an incomplete deck). One additional benefit of explicitly modeling object state is that it allows us to evaluate the impact of design decisions on the complexity of the abstract state space that must be considered when writing client code. For instance, in the example above, the state space is simple (three states) because of the decision to combine the deck initialization code with the shuffling code. Separating this behavior into distinct `initialize` and `shuffle` methods, or including a `sort` method, leads to a more complicated abstract state space for the object (see Figure 4.3 for an example).

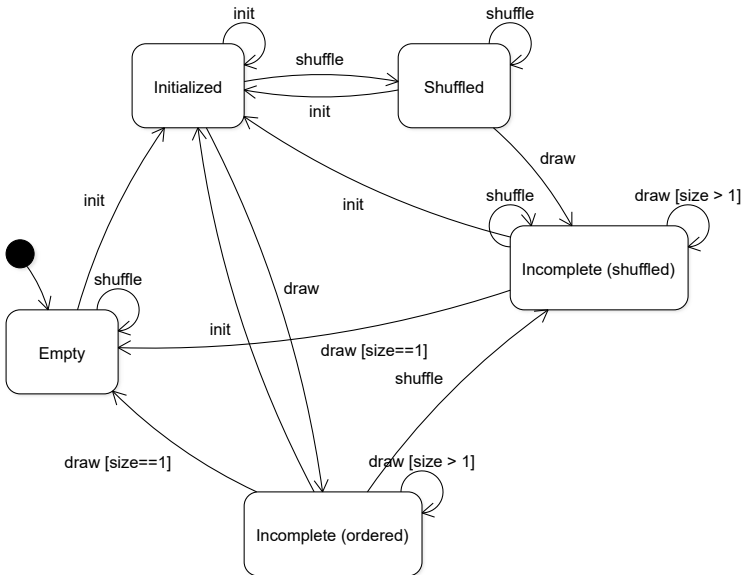


Fig. 4.3 State diagram for an instance of `Deck` where initialization and shuffling are separate operations

Objects with a complex life cycle are difficult to use, difficult to test (see Chapter 5), and their design and implementation is error-prone. A good design principle to avoid objects with complex life cycles is thus to *minimize the state space of objects* to what is absolutely necessary for the object to provide the required functionality. In practice, this means designing the class so that it is not possible to put

⁴ For which a better match is an activity diagram, which is, however, not covered in this book.

the object in an *unnecessary state*. An example of an unnecessary state is state *Initialized* in Figure 4.3, because there is no use for an unshuffled deck of cards. In some cases, eliminating some states from the life cycle of an object may seem like reducing the versatility of a class (*what if we need this one day?*). However, this kind of SPECULATIVE GENERALITY[†] is often not worth the cost. A related advice when designing classes is to avoid including any “accessory” field that does not capture the essential state of an object of the class. Examples of accessory fields include TEMPORARY FIELDS[†] (used only under limited conditions such as during initialization) and convenience fields that duplicate a value that can be obtained indirectly via another field. Such unnecessary fields make it more difficult to reason about the state space of objects of the class.

Code Exploration: JetUML · RecentFilesQueue

Abstract states that matter

Class `RecentFilesQueue` is a bounded list used to keep track of files recently opened by the application, with a maximum capacity (set at five files). Despite the fact that the class defines only one state-changing method (`add`), the life-cycle of objects of this class is rich with interesting cases. There are three meaningful abstract states that govern how the methods of the object behave: *Empty*, *Partially Filled*, and *Full*. In the *Empty* state, the *query* (i.e., non-state-changing) method `getMostRecentDirectory` returns a special value. In the *Filled* state, method `add` has an additional side-effect, namely to remove the last element in the list. Finally, through method `deserialize`, it is possible to transition from the initial state to any of the three abstract states mentioned above.

4.4 Nullability

One aspect of most programming languages that gets in the way of designing effective state spaces and life cycles for objects is the possibility to assign the value `null` to a variable of a reference type. In Java (and similar languages, such as C++), `null` is a special value that indicates, in a troublesome way, *the absence of value*. For example, if we assign `null` to a variable:

```
Card card = null;
```

we are in effect stating that the variable `card` is of a reference type `Card`, but that it refers to... nothing! This is problematic because variables of reference types are intended to be dereferenced. For example:

```
System.out.println(card.rank());
```

Because, with a reference to nothing, there is nothing to dereference, the result is the dreaded `NullPointerException`, the symptom of innumerable bugs in Java

software. Generally speaking, relying on the null reference can cause a lot of harm. In fact, the inventor of the null reference, Tony Hoare, is reported to have stated his regret at implementing this feature (see Further Reading). In terms of low-level design, null references are a problem because of how difficult it is to think about all possible program paths that may lead to a null dereference. Null references are also a liability for software design because of their inherent ambiguity. Depending on the situation, a null reference in the state of an object could be interpreted to mean:

1. That a variable is temporarily un-initialized, but is expected to become initialized in a different abstract state for the object. For example, in class `Deck`, we could assign to the field `aCards` the value `null` until the deck is shuffled;
2. That a variable is incorrectly initialized because the programmer overlooked a path through the code where the variable had to be initialized;
3. That the value is a flag that purposefully represents the absence of a useful value in the normal life cycle of an object;

To avoid unnecessarily enlarging and complicating the state space of an object with dangerous null references, a recommended practice is to design classes so that null references are simply not used. How to realize this goal in practice depends on whether there is a need to model the absence of a value for a variable or not. The Solitaire application is an example of a code base that does not use the null reference.

No Need to Model Absent Values

If it is possible to design a class to avoid any abstract state where a certain variable does not have a value, it is greatly desirable to design the class to prevent this eventuality. For example, normal playing cards *must* have a rank and a suit, so there is no reason to allow null references for either instance variable.

```
public class Card {
    private Rank aRank; // Should not be null
    private Suit aSuit; // Should not be null

    public Card(Rank pRank, Suit pSuit) {
        aRank = pRank;
        aSuit = pSuit;
    }
}
```

We can ensure that variables are not assigned a null reference by using either one of two approaches: input validation (Section 2.8), or design by contract (Section 2.9). With input validation, everywhere a variable can be assigned, we check whether the input value is a null reference and throw an exception if that is the case:

```
public Card(Rank pRank, Suit pSuit) {
    if (pRank == null || pSuit == null) {
        throw new IllegalArgumentException();
    }

    aRank = pRank;
    aSuit = pSuit;
}
```

With design by contract, we stipulate, using a precondition, that `null` is not a valid value for a variable and, optionally, check that the precondition is respected with an `assert` statement:

```
/**
 * @pre pRank != null && pSuit != null;
 */
public Card(Rank pRank, Suit pSuit) {
    assert pRank != null && pSuit != null;
    aRank = pRank;
    aSuit = pSuit;
}
```

In either case, if the `Card` constructor is the only place where `aRank` and `aSuit` can be assigned, we have effectively ensured that the value stored in either variable will not be a null reference.

Modeling Absent Values

In many situations, the domain concept we are trying to model will require that we make a provision for the fact that there may not be a value. As an example, let us consider a variant of class `Card` where an instance can also represent a *joker*. In many card games, a joker is a special card that has no rank and no suit. To identify a card as a joker, a simple approach is to add a `aIsJoker` field to its declaration:

```
public class Card {
    private Rank aRank;
    private Suit aSuit;
    private boolean aIsJoker;

    public boolean isJoker() {
        return aIsJoker;
    }

    public Rank rank() {
        return aRank;
    }

    public Suit suit() {
        return aSuit;
    }
}
```

Here the logic to determine whether a card represents a joker is simple enough, but what should we do with its rank and suit? As usual, different options are possible. We can review three:

- *Null references:* We could just ignore the advice offered in this section and assign `null` to `aRank` and `aSuit`. This means it would be possible to call (for example) `card.rank().ordinal()` on a joker, and get a `NullPointerException`.
- *Arbitrary values:* We could assign an arbitrary, meaningless value for the rank and suit of a joker (e.g., Ace of Clubs). However, this is both confusing and dangerous. A part of the code could erroneously request the rank of a joker, and receive the value `ACE`, which makes no sense. It is easy to imagine how tracking down this bug could be lengthy and annoying.
- *Special values of an enumerated type:* We could add an `INAPPLICABLE` enumerated value to both `Rank` and `Suit`, and assign these values to the corresponding fields for instances of `Card` that represent jokers. This solution also has some clear weaknesses. First, it is a conceptual abuse of the idea of enumerated types, where each value is enumerated. Conceptually `INAPPLICABLE` is not a valid value in the enumeration, but rather a flag that indicates that we do not have a value. Second, although we have four suits and 13 ranks, this solution will yield five and 14 enumerated values for each type, respectively. This discrepancy will muck up any code that relies on the ordinal values of these types (such as the initialization of a deck of cards), and introduce opportunities for off-by-one errors.

Fortunately, there are better solutions for avoiding the use of null references to represent absent values.

Optional Types

One solution is to use an *optional type*. In Java, optional types are supported by the `Optional<T>` library class. The `Optional` class is a generic type that acts as a wrapper for an instance of type `T`, which can be empty. To make a value of type `T` optional for a variable, we declare this variable to be of type `Optional<T>`. In our case:

```
public class Card {  
    private Optional<Rank> aRank;  
    private Optional<Suit> aSuit;  
}
```

To represent the absence of a value of the variable, we use the value returned by `Optional.empty()`. So, to create a constructor that instantiates a `Card` that represents a joker, we could have:

```
public class Card {
    private Optional<Rank> aRank;
    private Optional<Suit> aSuit;

    public Card() {
        aRank = Optional.empty();
        aSuit = Optional.empty();
    }

    public boolean isJoker() {
        return aRank.isEmpty();
    }
}
```

To create an instance of `Optional` that represents an actual value, we call `Optional.of(value)` if `value` is not (ever) expected to be null, and `Optional.ofNullable(value)` if `value` can be null (in which case `Optional.empty()` will be stored instead). To get the value wrapped by an instance of `Optional`, we call `get()`.

Using `Optional` in this way, we can both shed the dangerous use of null references and cleanly represent the absence of a value. The one main consequence, however, is that the two fields no longer have the types likely to be desired by the client code. While the client will probably be interested in working with values of type `Rank` and `Suit`, the fields of the class now store values of type `Optional<Rank>` and `Optional<Suit>`. To get around this issue, two main alternative are possible:

- *Change the interface of class `Card`* so that `rank()` and `suit()` return `Optional<Rank>` and `Optional<Suit>`, respectively. This requires client code to call `get()` everywhere the actual instance is needed, which is cumbersome.
- *Unwrap the optional within `rank()` and `suit()`*, which preserves the interface but requires clients to ensure that they do not call the methods on a card that represents a joker (something that could be specified using design by contract, for example). This last solution is starting to look a lot like the use of null references, but it is technically safer, because calling `get()` on an empty instance of `Optional` will raise an exception immediately when the value is misused, as opposed to potentially propagating a null reference through the execution of the code.

Code Exploration: JetUML · `TypeNode`

Using optional types.

Class `TypeNode` illustrates a classic scenario for using optional types. In JetUML, type nodes represent class and interface diagram elements in a class diagram. They can optionally be located in a package node. Correspondingly, the class declares a field `aContainer` to store a reference to the package node that contains the element. In this case, using an optional type as the type of the container is a great fit because it supports a feature that is actually optional. If

the type node is not contained in any package node, `aContainer.isEmpty()`. In this case, I implemented the access to the optional value using design by contract, by specifying that the getter method `getParent()` can only be called if the node has a parent (that is, a container).

The NULL OBJECT Design Pattern

There exists a second solution for avoiding the use of null references to represent absent values, which avoids the issue of unpacking wrapper objects. This solution uses a special object to represent the null value. For this reason, this idea is called the NULL OBJECT design pattern. Using a NULL OBJECT to represent a null value relies on polymorphism, so it is only applicable to situations where a type hierarchy is available. Because `Card` objects are not a subtype of any other user-defined type, we cannot use it to model a joker. To explore the NULL OBJECT pattern, let us consider a different scenario, where a `CardSource` in client code could be unavailable. `CardSource` is an interface that defines methods `draw()` and `isEmpty()` and that it is implemented by the `Deck` class (see Section 3.1).

The main idea of NULL OBJECT is to leverage polymorphism to create objects to represent the null (or absent) value. The methods of the subtype representing null objects can be called as usual, but their behavior is consistent with the absence of a value. Figure 4.4 illustrates NULL OBJECT applied to the context of handling potentially absent card sources. If a card source is unavailable, instead of assigning `null` to the corresponding variable, we create a new instance of `NullCardSource` and assign a reference to that object instead.

```
CardSource source = new NullCardSource();
/* ... */
if (!source.isEmpty()) {
    Card card = source.draw();
}
```

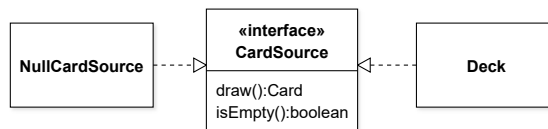


Fig. 4.4 The `CardSource` interface with support for the NULL OBJECT pattern

Let us now turn to the implementation of the methods of `NullCardSource`. The implementation of `isEmpty()` is straightforward: an absent card source does not have any cards, so we always return `true`:

```
public boolean isEmpty() {
    return true;
}
```

However, what should method `draw()` return for a `NullCardSource`? In this case it is helpful to revisit the declaration of the interface. In its interface declaration, method `draw` includes a precondition: `!isEmpty()`. However, because `isEmpty()` always returns `true` for instances of `NullCardSource`, method `draw()` should never be called. For this reason, it does not matter what it actually returns. To avoid returning anything altogether, we can throw an exception:

```
public Card draw() {
    assert !isEmpty();
    throw new IllegalStateException();
}
```

Because types for null objects are usually simple, we can also declare them as a constant in the corresponding interface using an anonymous class. The solution for the `CardSource` design context is the following:

```
public interface CardSource {

    CardSource NULL = new CardSource() {
        public boolean isEmpty() {
            return true;
        }

        public Card draw() {
            assert !isEmpty();
            throw new IllegalStateException();
        }
    };

    Card draw();
    boolean isEmpty();
}
```

With this solution, there is no longer a need for a separate `NullCardSource` class. Client code that must indicate an absent card source can simply use the reference available through `CardSource.NULL` instead. Because a `NULL CardSource` behaves just like any other card source, many special cases are avoided. In our case, for example, because all client code that works with a card source must check for emptiness first, obtaining a `NULL` card source is indistinguishable from obtaining an empty one.

The example described above shows an ideal application of `NULL OBJECT`, in the sense that we can define an implementation for the null object such that the client code does not need to check whether an object represents a null value or not. In certain situations, it can be necessary to test whether an object represents a null value or not. In such contexts, we can add an `isNull()` method to the interface. The behavior of this method is to return `true` for instances that represent null objects, and `false` for all other instances. A common scenario where an `isNull()` method is useful is when refactoring code to replace null values with a null object. In such a scenario, null checks on a variable, such as `if(value == null)`, can be directly replaced with `if(value.isNull())`. When no longer necessary, the `isNull()` method can then be retired.

Code Exploration: Solitaire · NullPlayingStrategy

Applying the NULL OBJECT pattern.

Class `NullPlayingStrategy` demonstrates two related applications of the NULL OBJECT pattern. First, the class itself is a null object value that represents the absence of a playing strategy. Injecting the `GameModel` class with this type of strategy is a clean way to disable the auto-play feature without flags or corner cases. Second, the class realizes the pattern by overriding method `getLegalMove` to return a null object for the `Move` class hierarchy. The null `Move` is declared in the initialization of `GameModel.NULL_MOVE`. Predictably, a null move represents the absence of an action in the game.

4.5 Final Fields and Variables

In Section 4.3, I argued that one useful principle to follow when designing a class is to keep the abstract state space for objects of the class to the minimum necessary for the objects of the class to provide the services expected of them. For example, a well-designed `Deck` class has three meaningful abstract states, not ten. Because object state is just an abstraction of the combination of values taken by the fields of an object, the way to realize the principle in practice is to limit the number of ways in which the field values can be updated. We already saw, in the previous section, how avoiding null references whenever possible can help us reach this goal. An even stricter constraint for keeping the abstract state space of objects to a minimum is to prevent changing the value of a field after initialization, so that the value of the field remains constant throughout the life of the object.

This constraint can be made explicit through the use of the `final` keyword placed in front of a variable declaration (which includes the declaration of instance variables). If we declare the fields `aRank` and `aSuit` to be `final` in class `Card`:

```
public class Card {
    private final Rank aRank;
    private final Suit aSuit;

    public Card(Rank pRank, Suit pSuit) {
        aRank = pRank;
        aSuit = pSuit;
    }
}
```

then the fields can be assigned a value *only once*, either in the initializer part of their declaration, or directly in the constructor (as in the example).⁵ Attempting to reassign the value of the field anywhere else in the code leads to a compilation error.

⁵ In practice, the field initialization code (the right-hand side of the equal sign in a field declaration) gets executed as part of the constructor call anyways.

The `final` keyword thus helps to limit the state space of an object, because any field marked as `final` can only take a single value. In the case of class `Card`, making both `aRank` and `aSuit` `final` renders objects of the class immutable, because the objects themselves are immutable and there are no other fields.

An important thing to keep in mind with the use of the `final` keyword is that, for reference types, the value stored in a variable is a *reference* to an object. So, although it is not possible to reassign a `final` field, it is certainly possible to change the state of the object referenced (if the object is mutable). Let us illustrate this point by making field `aCards` of class `Deck` `final`:

```
public class Deck {
    private final List<Card> aCards = new ArrayList<>();
}
```

A fresh instance of class `Deck` is shown in Figure 4.5.

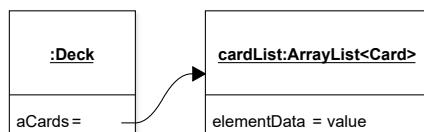


Fig. 4.5 Instance of class `Deck`

Because field `aCards` is `final` (something not visible on the diagram), we can be sure that the reference held in the field will *always* refer to the one `ArrayList` named `cardList` on the diagram. In other words, it will not be possible for this arrow to point anywhere else. However, we can (and need to) change the state of `cardList`, for example to initialize it with all the cards. Thus, although final fields can be very helpful in restricting the state space of an object to make it easier to understand the behavior of the object at run time, they do *not* make the referenced objects immutable.

The discussion above was concerned mainly with *instance* variables. However, local variables (including method parameters) can also be declared to be `final`. As opposed to fields, however, the case for making local variables `final` is much less clear because they are not long-lived.⁶ There is one fairly technical special case where local variables must not be reassigned (see Section 4.9), but even then the variable does not need to be explicitly marked with the `final` keyword.⁷ I occasionally declare a variable `final` to make my intent clear that the variable is not and should not be reassigned. This is only really useful for long and/or complex methods that may be a bit difficult to understand. Ideally, this should be a rare scenario, because well-designed methods are short and simple (and an overly `LONG METHOD`† is a recognized antipattern).

⁶ Local variables only exist for the duration of the execution of code in their scope.

⁷ Since Java 8, local variables that are not reassigned are considered *effectively final* by the compiler.

4.6 Object Identity, Equality, and Uniqueness

Three important concepts to keep in mind when designing object life cycles are those of *identity*, *equality*, and *uniqueness*.

Identity refers to the fact that we are referring to a particular object, even if this object is not in a variable. In terms of programming environments, the identity of an object could refer to its “memory location”, or “reference/pointer to”. However, in modern programming systems the memory management of objects is heavily abstracted, and for this reason it is best to think in terms of object identity. Most integrated development environments supply a convenient handle to represent an object’s identity. For example, in the Eclipse debugger this is represented by the object id.

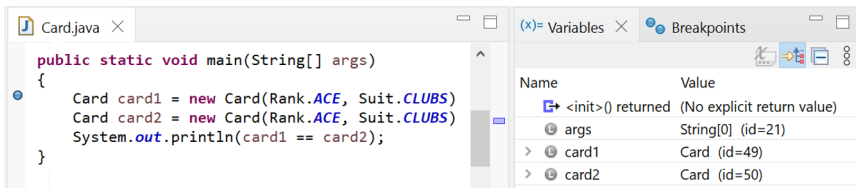


Fig. 4.6 Representation of object identity in the Eclipse debugger

In the small example of Figure 4.6 two `Card` objects are created, and consequently result in two distinct objects with two distinct identities, represented with internal object identifiers 49 and 50 (on the right, in the *Value* column). In the object diagram of Figure 4.7, the `main` method is represented as an object with two fields in place of local variables. The diagram shows how object identity corresponds to both object model elements and the references to these objects. If, for instance, a reference to the `Card` object with id 49 is added to a list, there will be two locations that refer to a single shared identity.

The last statement in the `main` method in Figure 4.6, is a reminder that in Java, the `==` operator returns `true` if the two operands evaluate to the same value. In the case of values of reference types, *the same value* means referring to the same object (identity). So here the statement returns `false` because, although both cards represent an Ace of Clubs, they are references to different objects.

The situation above, where two different `Card` objects represent the Ace of Clubs, illustrates the concept of *object equality*. In the general case, *equality* between two objects must be programmer-defined because the meaning of equality cannot always be inferred from the design of the object’s class. In very simple cases (like objects of class `Card`), one could say that two objects are equal if all their fields have the same value. However, for objects of more complex classes, this could be too strict. For example, if some objects cache values or have non-deterministic or unspecified internal representations, they could be “equal” in the practical sense, without having *precisely* the same value for each field, transitively. For example, two instances of

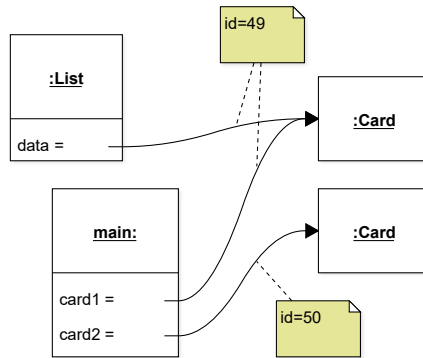


Fig. 4.7 Object identity example

a set abstract data type (such as Java's `Set`) must be equal if they have the same elements, even if internally the order in which these elements are stored is different.

For this reason, Java provides a mechanism to allow programmers to specify what it means for two objects of a class to be equal. This specification is realized by *overriding* the `equals(Object)` method of the `Object` class. The default implementation of the `equals` method defines *equality* as *identity*. In other words, if the `equals` method is not redefined for a class, `a.equals(b)` is practically the same as `a == b`.⁸ In many situations, like our example of playing cards, this is not what we need, and we must supply our own implementation of the `equals` method. Implementations of `equals` can usually follow this example as a template:

```

public boolean equals(Object pObject) {
    if (pObject == null) {
        return false; // As required by the specification
    }
    else if (pObject == this) {
        return true; // Standard optimization
    }
    else if (pObject.getClass() != getClass()) {
        return false;
    }
    else {
        // Actual comparison code
        return aRank == ((Card)pObject).aRank &&
            ((Card)pObject).aSuit == aSuit;
    }
}

```

I will revisit some of the details of the overriding mechanism in Chapter 7. For now, it suffices to say that if the `equals` method is *redefined* in a class, calling

⁸ Except in the case where `a == null`. The `==` operator will correctly compare null values, but if `a` is null, `a.equals(b)` will raise a `NullPointerException`.

`equals` on an object of this class will result in the redefined version being executed, and thus implement the custom definition of equality. In our case,

```
card1.equals(card2)
```

will return `true`.

A crucial constraint when overriding `equals` is that any class that overrides `equals` must also override `hashCode` so that the following requirement is respected:

If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result.

—Reference documentation for `Object#equals`

This constraint is necessary because, among other things, many classes of the Java class library's Collections framework rely on a combination of equality testing and an object's hash code for indexing objects in internal data structures.

A final consideration related to identity and equality is the concept of *uniqueness*. In our example code, we could rightfully wonder what is the point of tolerating duplicate objects that represent exactly the same card (e.g., Ace of Clubs). A sometimes useful property for the objects of a class is *uniqueness*. Objects of a class are unique if it is not possible for two distinct objects to be equal. If the objects of a class can be guaranteed to be unique, then we no longer need to define equality, because in this specific case, equality become equivalent to identity and we can compare objects using the `==` operator. Strict guarantees of uniqueness are almost impossible to achieve in Java due to mechanisms such as metaprogramming (see Section 5.4) and *serialization*.⁹ However, in practice, the use of a design pattern, presented below, and the conscious avoidance of metaprogramming and serialization, provide a good enough guarantee of uniqueness that can help simplify some designs.

Code Exploration: JetUML · Dimension

Methods `equals` and `hashCode` for records.

Type `Dimension`, first explored in Section 2.3, provided an example of a record type that yields objects that are not unique. Indeed, it is possible to create two distinct objects that represent exactly the same dimension (width and height). For this reason, we must rely on an implementation of the `equals` and `hashCode` methods to identify equal dimensions and manage them in collections. One benefit of record types is that an `equals` and `hashCode` method is automatically generated for them based on the values of the fields. Although there is no explicit declaration of these methods for `Dimension`, client code can still call `equals` on an object of type `Dimension`. The method will return `true` if both width and height are equal.

⁹ *Serialization* involves converting an object into a data structure that can be stored outside a running program and reconstructed later. The reconstruction of a serialized object normally leads to a copy of the serialized object being created.

4.7 The FLYWEIGHT Design Pattern

The FLYWEIGHT pattern provides a way to manage collections of shared objects that are usually simple and immutable. For example, `Card` objects in the Solitaire application are immutable and referenced in many different classes. Although sometimes used to address performance concerns, FLYWEIGHT is also valuable to ensure that objects of a class are unique.

The idea that underlies the pattern's solution template is to manage the creation of objects of a certain class, called the *flyweight* class. Instances of the flyweight class are called flyweight objects. The crucial aspect of the FLYWEIGHT is to control the creation of flyweight objects through an *access method* that ensures that no duplicate objects (distinct but equal) ever exist. The three main components necessary to realize this constraint are:

1. A *private constructor* for the flyweight class, so clients cannot control the creation of objects of the class;
2. A *flyweight store* that keeps a collection of flyweight objects;
3. An *access method* that returns the unique flyweight object that corresponds to some identification key. The access method typically checks whether the requested flyweight object already exists in the store, creates it if it does not already exist, and returns the unique object.

For example, we could decide to make the `Card` class a flyweight. Let us first consider the non-flyweight version:

```
public class Card {
    private final Rank aRank;
    private final Suit aSuit;

    public Card(Rank pRank, Suit pSuit ) {
        aRank = pRank;
        aSuit = pSuit;
    }

    /* Includes equals and hashCode implementations */
}
```

Instances of this class are clearly not unique, given that it is possible to use the constructor to create two instances that are distinct but equal:

```
Card card1 = new Card(Rank.ACE, Suit.CLUBS);
Card card2 = new Card(Rank.ACE, Suit.CLUBS);
System.out.println(String.format("Same?: %b; Equal?: %b",
    card1 == card2, card1.equals(card2)));
```

To implement Step 1 of the solution template, we simply change `public` for `private` in front of the constructor. This prevents client code from creating new `Card` instances arbitrarily. Now we need to figure out a way for code outside of class `Card` to get instances of the class. Before tackling this question (Step 3 in the solution template), let us create a store for flyweight `Card` instances (Step 2 in the

solution template). The two main decisions to make to realize this step are choosing a data structure to hold the instances, and deciding where to locate this structure in the code.

It is important to note that with design patterns, each application of the pattern can involve different implementation details. A solution template is just that: an overview of the main structures. For the FLYWEIGHT in particular, the implementation of the flyweight store and access method can exhibit much variability, depending on the details of the flyweight class.

In our case, because playing cards can be indexed in terms of two keys (rank and suit), I will store them in a two-dimensional array. As to where this array should be located, one option is to hold it as a static field in class `Card` so that we can make it private and use methods of class `Card` to access it. The following code shows the definition of the flyweight store and an implementation of its initialization.

```
public class Card {
    private static final Card[][] CARDS =
        new Card[Suit.values().length][Rank.values().length];

    static {
        for (Suit suit : Suit.values()) {
            for (Rank rank : Rank.values()) {
                CARDS[suit.ordinal()][rank.ordinal()] =
                    new Card(rank, suit);
            }
        }
    }
}
```

Because objects that represent playing cards are relatively small in number (52) and completely known in advance, I also chose to pre-initialize the flyweight store with a static initializer block.¹⁰ This implementation is only one example of a FLYWEIGHT implementation. Even for the same context (playing cards), many other alternatives are possible. For example, it would be possible to store flyweights in lists or hash tables.¹¹ With the current solution, accessing the collection with a correct index is guaranteed to produce the requested card. For example:

```
CARDS[Suit.CLUBS.ordinal()][Rank.ACE.ordinal()];
```

will return the (assumed unique) instance of `Card` that represents the Ace of Clubs.

The code above is only correct if it is placed within the scope of class `Card`, because `CARDS` is private. To grant access to cards to code outside the class, we need an access method. In our example, the implementation of this method is trivial:

```
public static Card get(Rank pRank, Suit pSuit) {
    assert pRank != null && pSuit != null;
    return CARDS[pSuit.ordinal()][pRank.ordinal()];
}
```

¹⁰ A block of code that executes once, when the class is first loaded into the run-time environment.

¹¹ The best choice is probably to use the `EnumMap` library type, but to get the point across with a minimum of explanation, the array-based solution is more accessible.

This method is static given that the flyweight store is static. The combination of the flyweight store and corresponding access method is sometimes referred to as the *flyweight factory*. In this context, I used static structures in the flyweight class as the flyweight factory. However, it is not the only option, as we could also create a separate class to fulfill this role (called, for example, `CardFactory`).

For flyweight objects that represent playing cards, the use of the pair (rank, suit) as the identification key is intuitive. In other scenarios, it can be less obvious what the identification key should be. For example, for an object of type `Person`, the key could be a name, an identification number, etc. In any case, the identification cannot be an instance of the flyweight object itself. In our example with cards, such an approach would look like:

```
Card card = Card.get(someCard); // INVALID
```

This would mean that to obtain a flyweight object of class `Card`, it would be necessary to already have that object. Because the only way to get a flyweight object should be through its access method, this scheme leads to an infinite cycle.

An important concern when implementing the FLYWEIGHT pattern is whether to pre-initialize the flyweight store, or whether to do this lazily, by creating objects as they are requested through the access method. The answer is context-dependent. In general, in cases where there exists a small and finite set of flyweights, it may make sense to pre-initialize them (as in the example). In other cases, additional logic must be added to the access method to check whether the object exists in the collection and, if not, create it based on the key. In this latter case, the access method needs to be able to access all the information it needs to create the flyweight instance that corresponds to the requested key. The following code shows the FLYWEIGHT-relevant portion of a version of the `Card` class where instances are lazily created:

```
public class Card {
    private static final Card[][] CARDS =
        new Card[Suit.values().length][Rank.values().length];

    public static Card get(Rank pRank, Suit pSuit) {
        if (CARDS[pSuit.ordinal()][pRank.ordinal()] == null) {
            CARDS[pSuit.ordinal()][pRank.ordinal()] =
                new Card(pRank, pSuit);
        }
        return CARDS[pSuit.ordinal()][pRank.ordinal()];
    }
}
```

Finally, the FLYWEIGHT pattern is especially convenient when used to manage *immutable* flyweight objects. Although it is technically feasible to apply the pattern to manage a collection of mutable objects, this approach can easily become error-prone. In any case, it is crucial to ensure that the portion of the flyweight objects' state that defines the objects' identity cannot be mutated.

Code Exploration: JetUML · Direction

Application of FLYWEIGHT with multiple access methods.

Class `Direction` represents a certain direction, in degrees, in a geometric plane. By design, degrees are represented as integers, with a maximum precision of one degree. This decision means that there can be at most 360 distinct directions to represent. I used the FLYWEIGHT to prevent a proliferation of `Dimension` objects, with numerous duplicates for the common values that represent the cardinal directions (north, south, etc.). One interesting aspect of this application of the pattern is that the flyweight factory provides multiple access methods. For example, `fromAngle` returns a dimension given an input angle, whereas `fromLine` finds the direction using a line in the plane as input, and `mirrored` finds the direction that is opposite to the input direction.

4.8 The SINGLETON Design Pattern

The SINGLETON design pattern provides a way to ensure that there is only one instance of a given class at any point in the execution of the code. The context for this design pattern is the need to manage an instance that holds, in one place, a cohesive amount of information that different parts of the code need. An example of a potential SINGLETON object in a card game would be the instance that represents the aggregated state of the game. This state could include the deck of cards and the various piles of cards in the game in progress. The solution template for SINGLETON involves three elements:

1. A *private constructor* for the singleton class, so clients cannot create multiple objects;
2. A *global variable* for holding a reference to the single instance of the singleton object.
3. An *accessor method*, usually called `instance()`, that returns the singleton instance. The accessor method is optional, because it is also possible to implement the pattern by declaring the global instance to be a public constant.

In a sample card game, a singleton object that encapsulates the aggregated state of the game, of class `GameModel`, could be implemented as follows:

```
public class GameModel {
    private static final GameModel INSTANCE = new GameModel();

    private GameModel() { /* ... */ }

    public static GameModel instance() {
        return INSTANCE;
    }
}
```

The SINGLETON pattern differs from FLYWEIGHT in that it attempts to guarantee that there is a *single instance of a class*, as opposed to *unique instances of a class*. Singleton objects are typically stateful and mutable, whereas flyweight objects are preferably immutable.

A typical mistake when implementing the SINGLETON pattern is to store a reference to an instance of the class in a static field called `INSTANCE` or something like it, without taking proper care to prevent client code from independently creating new objects. In this case, use of the *Singleton* name is harmfully misleading, because users of the code may rely on the fact that the class yields a single instance when in fact it does not.

The classic way to prevent instantiation is to make the class constructor private. However, in *Effective Java* [2], Bloch proposes a controversial trick, namely, to use an enumerated type (Item 3: *Enforce the singleton property with a private constructor or an enum type*). For example, to make a `GameModel` class a singleton, one could do:

```
public enum GameModel {  
    INSTANCE;  
  
    public void initializeGame() {}  
}
```

This technically works because the compiler will prevent the instantiation of enumerated types. Although this approach is presented as preferred in *Effective Java*, it is not without detractors. To me, this strategy uses a programming mechanism (enumerated types) for an intent other than originally designed and, as such, it can be confusing. Here the type `GameModel` is not a finite set of values representing different game models, which is what one would initially expect when seeing an `enum` designation. I thus recommend sticking to a private constructor to ensure the single-instance constraint.

Now that we know about the SINGLETON, I must mention that this pattern is controversial. While all design decisions involve trade-offs (see Section 1.1), in the case of the SINGLETON, the balance can often tip in favor of the disadvantages. First, a singleton is essentially a global instance, accessible from anywhere in the code. It is thus easy to make unprincipled use of this object, leading to numerous dependencies and code that is hard to understand. Second, singleton objects are difficult to test because they control their own life cycle and live for the entire duration of the application. Chapter 5 discusses testing in detail but, for now, it suffices to say that use of the SINGLETON makes a stateful object difficult to replace when necessary. In many cases, a good alternative for accessing a reference to a unique object is to use dependency injection (see Section 3.8). However, dependency injection does not provide a mechanism for preventing the creation of multiple instantiations of a class. This constraint must be respected with the assistance of methodical programming and documentation. What dependency injection helps achieve, however, is to propagate that single instance to the code that requires it.

Generally, what is important is to be able to recognize situations where only one instance of a class must be present in an application, and to be able to evaluate

different strategies for creating and managing this unique instance. In cases where dependency injection or another solution is fit for purpose, it may be preferable to employ it instead of using a `SINGLETON`. In some cases, however, the `SINGLETON` may turn out to be the preferable solution.

Code Exploration: Solitaire · `GameModel`

Managing a single instance without the `SINGLETON`.

In version 1.0 of Solitaire, I had implemented class `GameModel` as a singleton. Similar to the running example in this section, `GameModel` is the class used to create a single object that holds the complete state of the game. As the pattern dictates, the object was available as a global instance to any other part of the code. Later, I refactored `GameModel` to remove the part of the pattern that controls the life cycle of the object. The class is still meant to be instantiated only once, but in versions 1.2 and later, the life cycle is handled differently, to illustrate an alternative to `SINGLETON`. Specifically, `GameModel` is instantiated once when the application starts (in method `Solitaire#start`), and then injected into the objects that need a reference to it.

Code Exploration: JetUML · `ApplicationResources`

Using the `SINGLETON` for pervasive access to a service.

Class `ApplicationResources` in JetUML provides a way to obtain the various text strings that appear in the application (for example, button and menu labels). Following the practice of string externalization, these strings are not hard-coded, but stored in a separate configuration file. The class provides an example of the `SINGLETON`, but with a quirk: there is no `instance()` method. Instead, the field storing the singleton instance is called `RESOURCES` and is public. In this case, I compromised on the strict implementation of `SINGLETON`, mostly for cosmetic reasons. The singleton instance is referred to in over 100 locations in the code. Instead of the cumbersome `ApplicationResources.instance()` reference, I relied on Java's static import mechanism to import the name of the instance field. Because it would not be very meaningful to refer to just `INSTANCE`, however, I named the field `RESOURCES`. This way, the singleton instance of class `ApplicationResources` can be referred to anywhere in the code with the name `RESOURCES` (assuming the mention `import static ...ApplicationResources.RESOURCES;` is present). The fact that `ApplicationResources` is required in a very high number of locations is also a factor that justifies using `SINGLETON` instead of dependency injection.

4.9 Objects of Inner Classes

Inner classes are a special category of classes that are declared within the scope of an *enclosing* class and whose objects can discretely maintain a reference to other objects. This feature can affect their state space. Anonymous classes, introduced in Section 3.4, are a subcategory of inner classes.¹²

Inner Classes

Inner classes are used to provide additional behavior that involves an instance of the *enclosing* class, but which we do not want to integrate into the enclosing class. As an example, let us say we want the option to record how many times a certain `Deck` instance was shuffled. As usual, there are different ways of doing this. To illustrate how inner classes work, we define a `Shuffler` class as an inner class of `Deck`:

```
public class Deck {
    public void shuffle() { /* ... */ }

    public class Shuffler {
        private int aShuffles = 0;

        public Shuffler() {}

        public void shuffle() {
            aShuffles++;
            Deck.this.shuffle();
        }

        public int shuffles() {
            return aShuffles;
        }
    }
}
```

In this example, the first part of the declaration of class `Shuffler` looks normal: we declare a class `Shuffler` with a field `aShuffles` and a method `shuffle()`. However, things get interesting within the code of `Shuffler#shuffle()`, where we observe the statement `Deck.this.shuffle();`. Instances of an inner class automatically get a reference to the corresponding instance of their enclosing class (called the *outer instance*). The outer instance for an inner class is explicitly assigned to the inner instance when the inner instance is created. This can be confusing at first, so let us run through an execution:

```
Deck deck = new Deck();
Shuffler shuffler = deck.new Shuffler();
shuffler.shuffle();
```

¹² Java also supports *static nested classes*. Static nested classes are declared like inner classes, but with the `static` keyword placed before the class name. The main difference between static nested classes and inner classes is that static nested classes are *not* linked to an outer instance. Their purpose includes supporting encapsulation and code organization.

The first line creates a new instance of `Deck`, as usual. The second line calls the constructor of `Shuffler`. However, because `Shuffler` is an inner class of `Deck`, its instances must be associated with an instance of `Deck`. We specify this association by prefixing the constructor call with the name of a variable of type `Deck`, followed by a period. Within an inner class, the outer instance can be accessed through a qualified name that consists of the name of the class of the outer instance, followed by `this`. So, in our case, `Deck.this` refers to the outer instance of `Shuffler`. On the third line, it is the method `shuffle()` of the `Shuffler` instance that is called, but when this method executes, it then calls the `shuffle()` method of class `Deck` on the `deck` instance using `Deck.this`. Figure 4.8 illustrates the scenario with an object diagram.

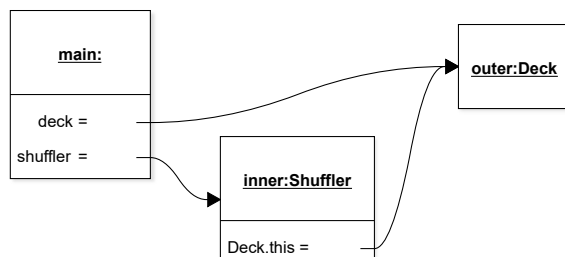


Fig. 4.8 Object graph for an inner class

With this design, the deck can be shuffled through the `shuffler` instance, which will record how many times the method was invoked. It is also possible to shuffle the deck without going through the `shuffler` instance, in which case the field `aShuffles` will not be incremented.

An important consequence of the inclusion of a reference to an outer instance is that the concrete state space of objects of inner classes adds to that of the outer instance. Although this can technically lead to increased complexity, it does not have to be so. With good design, the *abstract* state space of objects of inner classes can remain independent of the state of the outer instance. This is the case for the `Shuffler` class, for which the state of the outer `Deck` instance does not influence how we use the `Shuffler` instance.

Anonymous Classes and Variable Capture

Anonymous classes are a subcategory of inner classes. In consequence, they also have implicit access to additional state information through a reference to their outer instance.¹³ Let us consider the following code for a factory method that creates a

¹³ Java also supports *local classes*, which are named but declared within a code block such as a method declaration. Because local classes are rarely used, this section focuses on anonymous classes. However, local classes work in a similar way.

`Comparator<Deck>` instance that compares two decks in terms of the number of cards of a given rank that they have.

```
public class Deck {
    public static Comparator<Deck> createRankComparator(Rank pRank) {
        return new Comparator<Deck>() {
            public int compare(Deck pDeck1, Deck pDeck2) {
                return countCards(pDeck1) - countCards(pDeck2);
            }

            private int countCards(Deck pDeck) {
                /* returns the number of cards in pDeck with pRank */
            }
        };
    }
}
```

For example, in the code below, to see whether `deck1` contains more kings than `deck2`, we could do:

```
Comparator<Deck> comp = Deck.createRankComparator(Rank.KING);
int result = comp.compare(deck1, deck2);
```

This solution is an example of a factory method used to create a function object of type `Comparator`, as explained in Section 3.4. The code is relatively unexciting, except perhaps for one intriguing observation. Upon closer inspection, it appears that the code of method `compare` *declared inside the anonymous class* has access to the parameter `pRank` of `createRankComparator`, which is a *separate method in a separate class*. What could `pRank` possibly refer to when the code is running? Once the `createRankComparator` method returns an object, this object has its own life cycle that is independent from that of the `Deck` object. Yet this is legal, compilable code, that actually works and does what we want.

Because referring to variables in the parent method from an anonymous class is such a useful programming idiom, it is supported by the language. To make this work, when the compiler creates the definition of the anonymous class, it also (invisibly) adds *fields* to the anonymous class, and copies references to each of the local variables referenced in the code of the anonymous class's method into a field. Thus, once an object of the anonymous class is created, the references to the local variables are now stored in fields of the same name in the anonymous class. The object diagram of Figure 4.9 illustrates the outcome.

In this diagram, the factory method is represented as a separate object with field `pRank` used to represent its parameter. This method returns a new object of an anonymous class. So that the `compare` method can still refer to the `pRank` parameter, a field `pRank` is created in the instance of the anonymous comparator, and the value of `pRank` is copied to it. A method definition together with references to its local environment variables is sometimes called a *closure*.¹⁴ As the object diagram shows, it

¹⁴ In Java, anonymous classes and lambda expressions are not closures in the strict sense because they cannot modify the variables they reference. However, because they are as close as we can get to closures in Java, I employ the term to refer to methods that capture some of the values in non-local variables, as in this case.

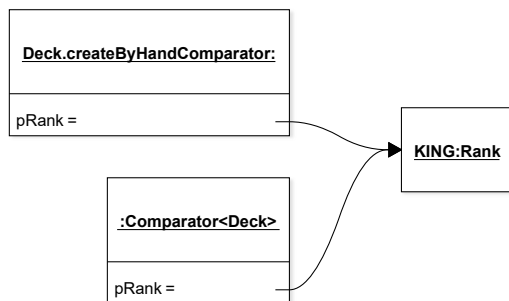


Fig. 4.9 Example of closure at run time

should be clear that closures can lead to shared references between object instances. To prevent unexpected behavior, Java prevents referencing external variables that are reassigned within anonymous classes.¹⁵

Code Exploration: Solitaire · GameModel

Instances of anonymous classes used as closures.

In class `GameModel`, field `aDiscardMove` represents a specific type of move in the game, to discard a card from the deck. How this works with the rest of the code is the topic of a later chapter. However, the initialization of the field provides an example of how instances of anonymous classes retain a reference to their parent class. By instantiating an anonymous subtype of interface `Move`, the initialization of the field has to provide an implementation for the three methods of the interface. Part of the explanation is to be found in the code of method `perform`, which actually performs the discard move. In the code, we see references to three fields: `aDiscard`, `aDeck`, and `aMoves`. These are *not* fields of the anonymous class, but fields of its enclosing class, `GameModel`.

Insights

This chapter defined object state, argued that keeping track of all the different abstract states an object can go through can be difficult, and proposed a number of techniques for designing classes whose objects have simple and well-structured life cycles.

- For stateful classes, consider using state diagrams to reason systematically about the abstract states of the object of the classes and their life cycle;

¹⁵ Prior to Java 8, local variables referenced within anonymous classes had to be declared `final`. With Java 8 the compiler can infer variables to be *effectively final* without the keyword.

- Minimize the number of meaningful abstract states for objects of a class: ensure it is not possible to put the object in an unnecessary state, and avoid introducing TEMPORARY FIELDS[†] or convenience fields;
- Avoid using null references to represent legal information in objects and variables; consider using optional types or the NULL OBJECT pattern to represent absent values, if necessary;
- Consider declaring instance variables `final` whenever possible;
- Be explicit about whether objects of a class should be unique or not;
- If objects are not designed to be unique, override the `equals` and `hashCode` methods; if objects should be unique, consider using the FLYWEIGHT pattern to enforce uniqueness;
- Consider an explicit structure, such as SINGLETON or dependency injection, for managing the object of classes that should have only one shared instance.
- Remember that additional data can be attached to instances of inner classes, either in the form of a reference to an instance of an outer class, or as copies of local variables captured in a closure.

Further Reading

The Gang of Four book [7] has the original treatment of the FLYWEIGHT and SINGLETON patterns. Chapter 3 of *Refactoring: Improving the Design of Existing Code* [4] mentions the TEMPORARY FIELD[†] and LONG METHODS[†] antipatterns. The entry *Introduce Null Object* in Chapter 8 discusses the idea of NULL OBJECT.

Chapter 10 of *Java 8 in Action* [16] is entitled *Using Optional as a better alternative to null*, and provides more details and examples on the use of the `Optional` type. This is also where I found the anecdote about Tony Hoare. Item 55 in *Effective Java* [2] provides useful insights on when and how to use optional types.

The API documentation for `Object#equals(Object)` and `Object#hashCode()` provide additional information on the meaning of equality in Java. The section on nested classes in the Java Tutorial [11] is a good reference for additional coverage of this topic.